7/29 3회차 스터디



▼ 예습 해오기!

예습은 깃허브 환경세팅 파트 해오시고 시간 되시면 이론파트 읽어오시면 됩니다.

https://github.com/likelion-server-study/WEEK3

데이터베이스를 처음 공부해 본다면 아래 영상이랑 책 데이터베이스 파트 보고 오시면 도움 될 거 같습니다.

https://www.youtube.com/watch?v=IAMdPn3YCG4&t=20s

https://www.youtube.com/watch?v=ZVuHZ2FjkI4

도커에서 postgreSQL 사용법 시간 되시면 보고 오시면 됩니다.

https://diary-developer.tistory.com/20

이번 스터디는 데이터베이스 스터디가 아니라 서버 스터디라, 도커 위에서 PostgreSQL 돌리는 것으로 준비했습니다. 만약, 도커 컨테이너가 아니라 로컬에서 직접 sql쓰면서 진행해보고 싶으시면 OS 맞춰서 postgreSQL앱 다운 받으시고 GUI 들어가서 직접 sql 명령어 제어해볼 수도 있습니다.

🐘 PostgreSQL 및 트랜잭션

https://github.com/likelion-server-study/WEEK3

▼ 4주차 미리보기

다음 4주차에는 3주차에서 연결한 PetClinic과 PostgreSQL 앞에 Nginx를 추가하고, 세 컨테이너를 Docker Compose로 함께 실행합니다.

지금까지는 사용자가 PetClinic의 8080 포트로 직접 접속했습니다. 이번 주차부터는 PetClinic을 외부에서 직접 접근할 수 없게 만들고, Nginx를 통해서만 접속하도록 구조

를 변경합니다.

```
사용자
  ↓ localhost:80
Nginx
  ↓ app:8080
PetClinic
  ↓ postgres:5432
PostgreSQL
```

Nginx가 요청을 PetClinic으로 전달하는 과정뿐만 아니라, 정적 파일 캐싱과 과도한 요청 제한 등 서버 앞단에서 할 수 있는 작업을 함께 확인합니다.

4주차에 확인할 문제

- PetClinic·PostgreSQL·Nginx를 각각 실행하지 않고 함께 관리할 수 있는가

  → Docker Compose를 이용해 세 컨테이너를 하나의 구성으로 실행하고 종료합니다.

- 사용자가 PetClinic에 직접 접근하지 못하게 할 수 있는가

  → PetClinic의 8080 포트를 외부에 공개하지 않고, Nginx의 80번 포트만 공개합니다.

  → `localhost:8080`으로는 접속할 수 없지만 `localhost:80`으로는 PetClinic이 열리는지 확인합니다.

- Nginx는 요청을 PetClinic으로 어떻게 전달하는가

  → Nginx가 `localhost:80`으로 들어온 요청을 Docker 내부의 `app:8080`으로 전달하도록 리버스 프록시를 구성합니다.

- 정적 파일을 매번 PetClinic에서 받아와야 하는가

  → CSS·이미지 같은 정적 파일을 Nginx가 임시로 저장하고 직접 응답하도록 캐시를 적용합니다.

  → 같은 파일을 두 번 요청하여 첫 번째 요청은 `MISS`, 두 번째 요청은 `HIT`가 나오는지 확인합니다.

- 짧은 시간에 요청이 너무 많이 들어오면 어떻게 막을 수 있는가

  → Nginx에서 일정 시간 동안 허용할 요청 수를 설정합니다.

  → 여러 요청을 동시에 보내 정상 요청은 `200 OK`, 제한을 넘은 요청은 `429 Too Many Requests`가 나오는지 확인합니다.

- PetClinic 서버가 멈추면 어떤 결과가 나타나는가

→ Nginx는 실행 중이지만 요청을 전달할 PetClinic이 없으면 `502 Bad Gateway`가 발생하는지 확인합니다.

→ 로드밸런싱과 장애 시 다른 서버로 요청을 보내는 실습은 5주차에 진행합니다.

다음 주에 사용하는 구성

구성 요소	역할
Docker Compose	PetClinic·PostgreSQL·Nginx를 하나의 구성으로 관리
Nginx	사용자의 요청을 가장 먼저 받는 공통 입구
Reverse Proxy	Nginx가 받은 요청을 PetClinic으로 전달
Static Cache	CSS·이미지 응답을 Nginx가 임시로 저장
Rate Limiting	짧은 시간에 과도하게 들어오는 요청을 제한
PetClinic	화면과 서비스 기능 처리
PostgreSQL	PetClinic 데이터 저장

▼ 간단한 진행 흐름

1. 기존 실행 상태 확인

로컬에서 Gradle로 실행 중인 PetClinic이 있다면 `Ctrl + C`로 먼저 종료합니다.

```
docker image ls | grep spring-petclinic
docker compose version
```

2. Docker Compose와 Nginx 설정 준비

다음 구성 파일을 사용합니다.

```
docker-compose.yml
docker-compose.override.yml
compose.app.ymlcompose.nginx.yml
nginx/nginx.conf
```

PetClinic에는 외부 포트 연결을 설정하지 않고 Docker 내부의 `8080` 포트만 사용합니다.

Nginx에는 다음 설정을 적용합니다.

- 외부의 80번 포트에서 요청 받기
- 일반 요청을 `app:8080`으로 전달하기
- CSS·이미지 같은 정적 파일 캐싱하기
- 일정 횟수를 초과한 요청을 `429`로 제한하기
- 접근 결과와 오류를 로그로 기록하기

3. 최종 Compose 설정 확인

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f compose.app.yml \
  -f compose.nginx.yml \
  config
```

여러 Compose 파일이 최종적으로 어떻게 합쳐지는지 확인합니다.

4. 전체 컨테이너 실행

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f compose.app.yml \
  -f compose.nginx.yml \
  up -d
```

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f compose.app.yml \
  -f compose.nginx.yml \
  ps
```

5. PetClinic 직접 접근 차단 확인

```
curl -i http://localhost:8080
```

PetClinic 포트를 외부에 공개하지 않았으므로 접속에 실패해야 합니다.

Nginx를 통해 다시 접속합니다.

```
curl -i http://localhost
```

이번에는 Nginx가 요청을 PetClinic으로 전달하므로 `200 OK`와 PetClinic 응답이 나와야 합니다.

```
localhost:8080 → 직접 접근 실패
localhost:80   → Nginx를 통해 접근 성공
```

6. 정적 파일 캐시 확인

같은 CSS 파일을 두 번 요청합니다.

```
curl -I http://localhost/resources/css/petclinic.css
curl -I http://localhost/resources/css/petclinic.css
```

응답 헤더를 비교합니다.

```
첫 번째 요청: X-Cache-Status: MISS
두 번째 요청: X-Cache-Status: HIT
```

7. 과도한 요청 제한 확인

짧은 시간에 여러 요청을 동시에 보냅니다.

```
seq 1 30 | xargs -P20 -I{} \
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost/
```

설정한 요청 수 이내에서는 `200`이 나오고, 제한을 넘은 일부 요청에서는 `429`가 나오는지 확인합니다.

```
200 → Nginx가 PetClinic으로 전달한 요청
429 → Nginx가 앞단에서 제한한 요청
```

8. PetClinic 장애 확인

PetClinic 컨테이너만 중지합니다.

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f compose.app.yml \
  -f compose.nginx.yml \
  stop app
```

다시 접속합니다.

```
curl -i http://localhost
```

Nginx는 실행 중이지만 PetClinic이 멈췄기 때문에 `502 Bad Gateway`가 나오는지 확인합니다.

실습 후 PetClinic을 다시 실행합니다.

```
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f compose.app.yml \
  -f compose.nginx.yml \
  start app
```

성공 기준

- PetClinic·PostgreSQL·Nginx가 Docker Compose로 함께 실행된다.
- `localhost:8080`으로는 PetClinic에 직접 접근할 수 없다.
- `localhost:80`에서는 Nginx를 통해 PetClinic에 접속할 수 있다.
- 같은 정적 파일을 두 번 요청하면 `MISS` 다음 `HIT`가 표시된다.
- 과도한 요청을 보내면 일부 요청이 `429 Too Many Requests`로 제한된다.
- PetClinic을 중지하면 Nginx에서 `502 Bad Gateway`가 발생한다.
- Nginx 로그를 통해 요청 처리 결과를 확인할 수 있다.